Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Lecture 4

# Design and analysis of Recursion Algorithms

Dr Rahaf Ghazal

2025-2026

الخوارزميات العودية ( Recursive Algorithms )

- خوارزميات التتبع الخلفي - التراجعية (Backtracking algorithms)
- خوارزميات البرمجة الديناميكية (Dynamic Programming Algorithms)
- خوارزميات التفريق ثم الحل (Divide and Conquer Algorithms)
- الخوارزميات الجشعة والأنانية (Greedy Algorithms)

# Define and Design the Recursion Algorithms

# Introduction

- Recursion is a problem-solving approach in which a problem is solved using repeatedly applying the same solution to smaller instances.(In other words, the algorithm calls to itself)
- It can be described loosely as follows:
  - If the given instance of the problem is small or simple enough, just solve it**( Base Case).**
  - Otherwise, reduce the problem to one or more simpler instances of the same problem**(recursion definition).**

# Examples on recursion problems

1- تعريف الأعداد الطبيعية

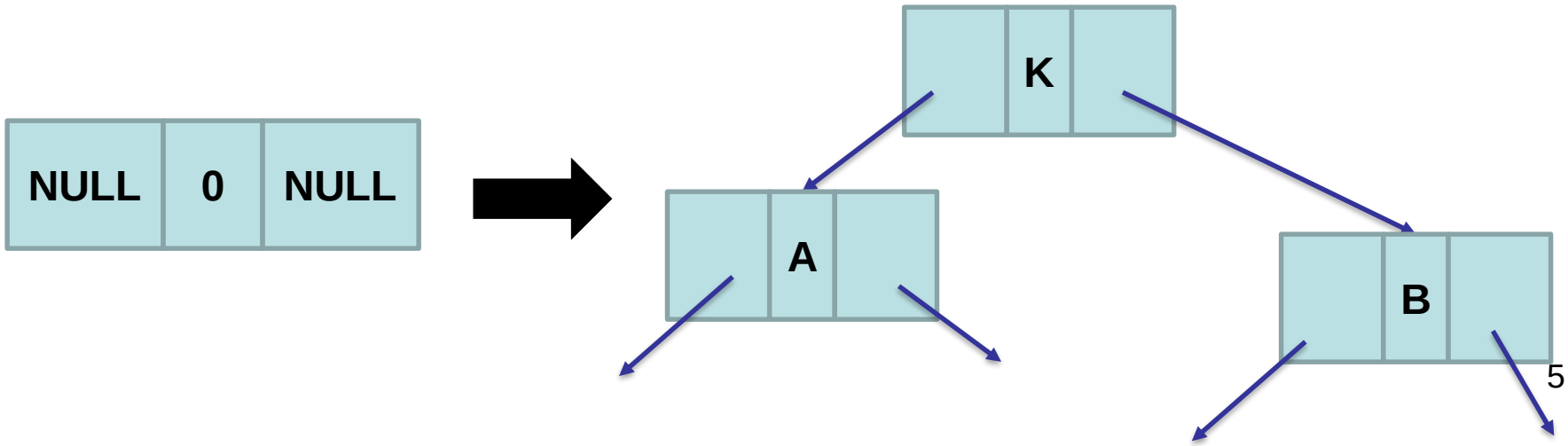
a- الصفر هو عدد طبيعي

b- إذا كان $n$ عدداً طبيعياً فإن $n + 1$ هو عدد طبيعي.

2- تعريف بنية الشجرة الثنائية:

a- الشجرة 0 لا تحوي أية عقدة نسميها الشجرة الفارغة.

b- إذا كانت $A$ و $B$ شجرتين ثنائيتين فإن $(k,A,B)$ هو شجرة ثنائية جذرها $k$ وشجرتها الجزئية اليسارية $A$ وشجرتها الجزئية اليمينية $B$.



# Examples on recursion problems

3- تعريف العاملي $y = n!$

a- $0! = 1$

b- إذا كان $n > 0$ فإن $y = n! = n(n-1)!$

pseudocode

```
1. Start.
2. function factor( int : n ).
   2.1. if( n = 0 ).
      2.1.1. return 1.
   2.2. else.
      2.2.1. return n * factor(n – 1 ).
3. End.
```

4- تعريف متتالية فيبوناشي Fibonacei

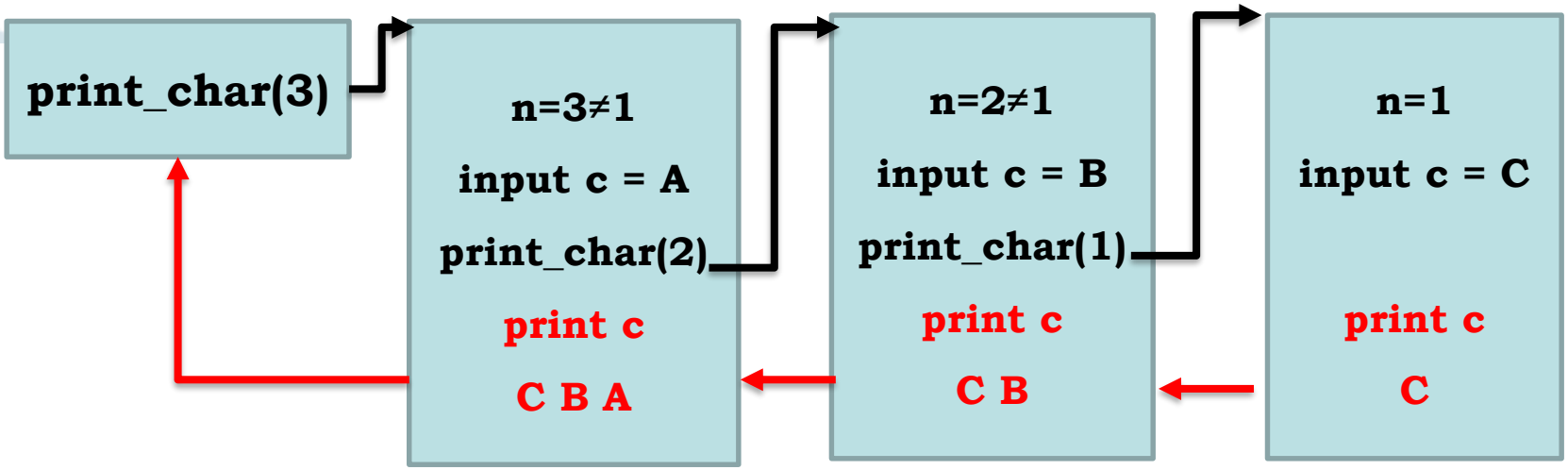
a- $Fib(1) = 1 , Fib(0) = 0$

b- إذا كان $n > 1$ فإن $Fib(n) = Fib(n-1) + Fib(n-2)$

# Exercise 1

Design a recursive algorithm to input a set of characters from the keyboard and print them on the screen in reverse order of input.

1. Start.
2. function print_char( int : n ).
   - 2.1. if( n = 1 ).
     - 2.1.1. input c.
     - 2.1.2. print c.
   - 2.2. else.
     - 2.2.1. input c.
     - 2.2.2. print_char( n – 1 ).
     - 2.2.3. print c.
3. End.

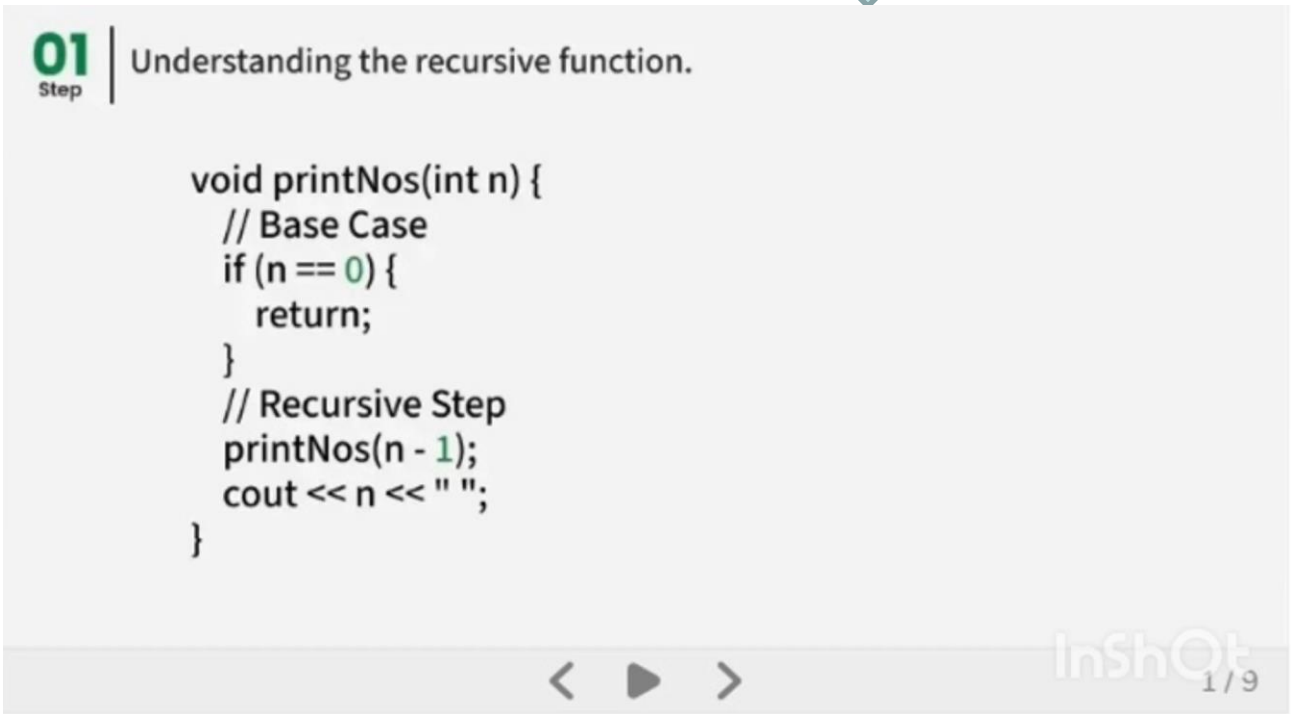


## Exercise 2

Design a recursive algorithm to Print numbers from 1 to n. Where n is a completely positive integer number.

1. **Start**
2. **Function printNos(int n)**
   - **2.1. if (n==0)**
     - **//stop the recursion calls and ends the execution of function**
   - **2.2. else**
     - **2.2.1. printNos(n-1)**
     - **2.2.2 print n**
3. **End**

execution



# Homework

1) حاول أن توجد الخوارزمية التكرارية لحساب عاملي عدد موجب صحيح
2) ادرس مسألة أبراج هانوي وخوارزميتها العودية Hanoi Tower
3) اكتب خوارزمية عودية لإيجاد القاسم المشترك الأكبر بين عدديدن صحيحين
4) اكتب خوارزمية عودية لحساب جداء عددين صحيحين

# Analyze the Recursion Algorithms

# Introduction

- The time-complexity analysis of a recursive algorithm is usually determined by solving a recurrence relation, which is a recursive definition of a function.
- للقيام بتحليل كلفة الخوارزميات العودية نتجه إلى الحصول على علاقات عودية حيث تفيد بتعريف حد ما بدلالة الحدود السابقة له، فهي علاقة تحتوي على حالة قاعدية Base case وحالة عودية recursive case، إذاً لتحليل كلفة الخوارزميات العودية:

1. نوجد العلاقة العودية المقابلة للخوارزمية.
2. نحلل هذه العلاقة لتقييم التعقيد الخوارزمي.

# 1) The recurrence relation:

Steps to determine the recursive relationship:

I. Specify the base case time (usually O(1)).
II. Specify the number of recursive calls.
III. Specify the size of the subproblem (is it n-1 ? Or n/2 ?).
IV. Add the cost of other operations (merging or processing).

**Examples:**

Example 1: Let's have the sequence : 3, 7, 11, 15, …

**Mathematical Equation:** $a_n = a_{n-1} + 4$ , n>1

**Base case:** $for \quad n = 1 \quad a_1 = 3$

**Pattern:** Add 4 to the previous term

**Recurrence relation:** T(n)= T(n-1)+C

---

Example 2: Fibonacci with recursion

**Mathematical Equation:** Fib(n)=fib(n-1)+fib(n-2) , n>1

**Base case:** n if n=0, n=1

**Recurrence relation:** T(n)=T(n-1)+T(n-2)+C

## 2) Solve the recurrence relation:

There are various techniques to solve recurrences.

1) Iterative method ( الطريقة التكرارية )
2) Recursion tree method ( طريقة الشجرة العودية )
3) Master method ( الطريقة العامة )
4) Substitution method ( طريقة التعويض )

# Iterative method – Example 1

let's have the following pseudocode. Study the time complexity of it.

```
Algo fact(n) {
    if n == 1
        return 1;
    else
        return n * fact(n-1);
}
```

Writing the recurrence relation: **1**

$$T(n) = \begin{cases} T(n-1) + C & \text{if } n > 1 \\ C & \text{if } n = 1 \end{cases}$$

Expand the original relation to find an equivalent general expression in terms of the number of expansions

2

$$T(n) = T(n-1) + C$$
$$T(n) = T(n-2) + 2C$$
$$T(n) = T(n-3) + 3C$$
$$\vdots$$
$$T(n) = T(n-k) + kC$$

3

Find a closed-form expression by setting the number of expansions to a value which reduces the problem to abase case:

- n-k=1 means k=n-1
- So T(n) = T(1)+ (n-1)c = c+ n.c –c = n.c
- So T(n) is O(n)

# Iterative method – Example 2

let's have the following recurrence relation. Solve it by iterative method to deduce its time complexity.

$$T(n)= 2T(n/2)+n \;\; ; \;\; \text{base case is } T(1)=c$$

$$\Rightarrow T(n) = 2(2T(n/4)+n) +n = 4T(n/4)+3n$$

$$\Rightarrow T(n) = 4(2T(n/8)+n) +n = 8T(n/4)+5n$$

.

.

.

$$\Rightarrow T(n) = 2^K T(n/2^k)+(k+1)n$$

$$\Rightarrow \ldots. \Rightarrow T(n) \text{ is } O(n\lg(n))$$

# Recursion tree method

- In this method, a recurrence relation is converted into recursive trees. Each node represents the cost incurred at various levels of recursion. To find the total cost, costs of all levels are summed up.

- Steps to solve recurrence relation using recursion tree method:

1) Draw a recursive tree for given recurrence relation
2) Calculate the cost at each level and count the total no of levels in the recursion tree.
3) Count the total number of nodes in the last level and calculate the cost of the last level
4) Sum up the cost of all the levels in the recursive tree

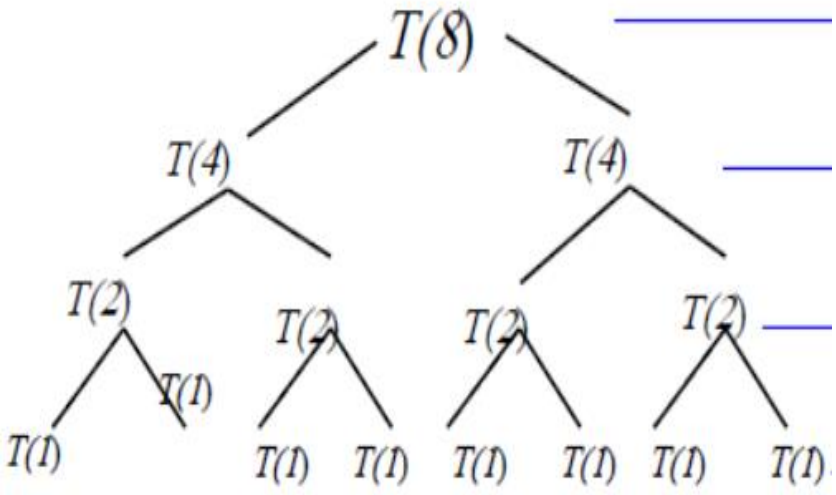
# Recursion tree method – Example 1

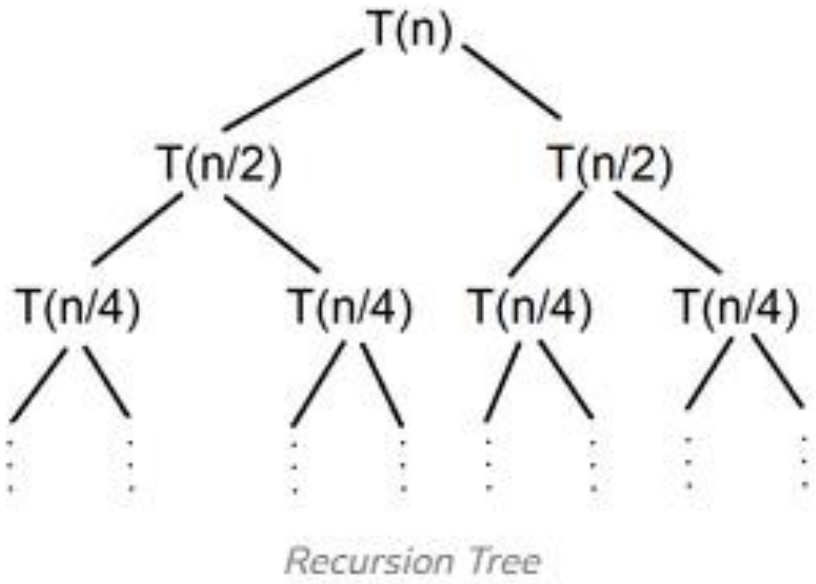
let's have the following recurrence relation. Solve it by Recursion tree method to deduce its time complexity.

**T(n) = 2T(n/2) + c ; base case is T(1)=c**

- **Step 1:** Draw a recursive tree

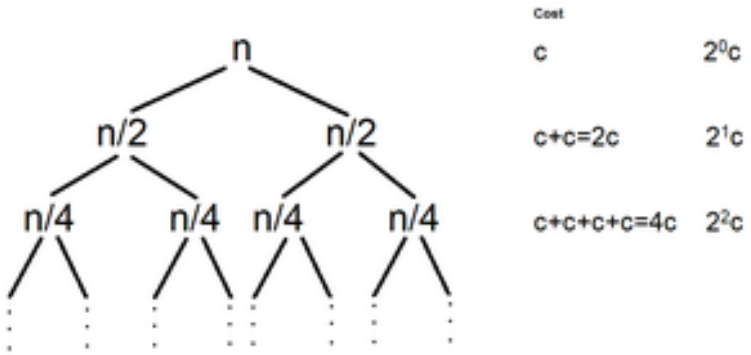
مثال




Recursion Tree

- **Step 2:** Calculate the work done or cost at each level and count total no of levels in recursion tree




*Recursive Tree with each level cost*

**Count the total number of levels -**

Choose the longest path from root node to leaf node

$$n/2^0 \longrightarrow n/2^1 \longrightarrow n/2^2 \longrightarrow \ldots\ldots\ldots \longrightarrow n/2^k$$

Size of problem at last level = $n/2^k$

At last level size of problem becomes 1

$$n/2^k = 1$$
$$2^k = n$$
$$\boldsymbol{k = \log_2(n)}$$

**Total no of levels in recursive tree = k +1 = $\log_2(n) + 1$**

- **Step 3:** Count total number of nodes in the last level and calculate cost of last level

*No. of nodes at level 0 = $2^0 = 1$*

*No. of nodes at level 1 = $2^1 = 2$*

.....

*No. of nodes at the (k+1)th level (or last level) = $2^k = 2^{log_2(n)} = n^{log_2(2)} = n$*

*Cost of sub problems at level last level = n x $\Theta(1) = \Theta(n)$*

- **Step 4:** Sum up the cost all the levels in recursive tree

*T(n) = c + 2c + 4c + ---- ((no. of levels-1) times) + last level cost*

*= c + 2c + 4c + ---- ($log_2(n)$ times) + $\Theta(n)$*

*= c(1 + 2 + 4 + ---- ($log_2(n)$ times)) + $\Theta(n)$*

*1 + 2 + 4 + ----- ($log_2(n)$ times) --> $2^0 + 2^1 + 2^2$ + ----- (log2(n) times) --> Geometric Progression(G.P.)*

*Using GP Sum Formula, we get the sum of the series as n*

*= c(n) + $\Theta(n)$*

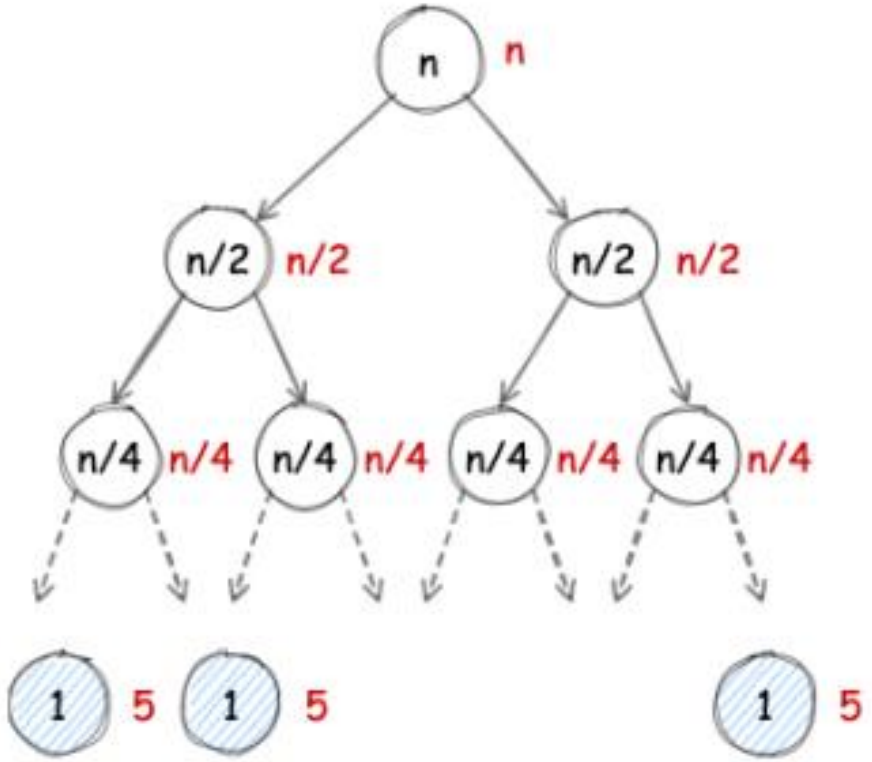
Thus, ***T(n) = $\Theta$(n)***

# Recursion tree method – Example 2

let's have the following recurrence relation. Solve it by Recursion tree method to deduce its time complexity.

$$T(n) = 2T(n/2) + n \;;\; \text{base case is } T(1)=5$$

**The solution:**




$$T(n) = O(n\log_2(n))$$

# Master method

- Is a tool used to solve recurrence relations that arise in the analysis of divide-and-conquer algorithms, in terms of asymptotic notations.
- Steps to solve recurrence relation using Master method:

1) First, we need to have an algorithm runtime(recursive relation) in the following format:

***T(n) = aT(n/b) + f(n)***

*where n = size of the problem*
*a = number of subproblems in the recursion and a >= 1*
*n/b = size of each subproblem*
*f(n) = cost of work done outside the recursive calls like dividing into subproblems and cost of combining them to get the solution.*

*Not all recurrence relations can be solved with the use of the master theorem i.e. if*

- *T(n) is not monotone, ex: T(n) = sin n*
- *f(n) is not a polynomial, ex: T(n) = 2T(n/2) + $2^n$*

# Master method

- Steps to solve recurrence relation using Master method:

2) Second, we will give the solution according to relationship between a,b: and form of f(n) function.

$T(n) = O(n)$ if $a<b$

$T(n) = O(n\log n)$ If $a=b$

$T(n) = O(n^{\log_b a})$ if $a>b$

Here we study a special form of f : $f(n) = cn$

There are the following three cases:

- If $f(n) = O(n^c)$ where $c < \text{Log}_b a$ then $T(n) = \Theta(n^{\text{Log}_b a})$
- If $f(n) = \Theta(n^c)$ where $c = \text{Log}_b a$ then $T(n) = \Theta(n^c \text{Log } n)$
- If $f(n) = \Omega(n^c)$ where $c > \text{Log}_b a$ then $T(n) = \Theta(f(n))$

# Master method – Examples

let's have the following recurrence relations. Solve it by Master method to deduce its time complexity.

## Example 1

$T(1) = 0$

$T(n) = 4T(n/2) + cn$ For n=2

$a = 4; b = 2; a > b$

$T(n) = O(n^{\log_b a})$ if $a > b$

$T(n) = O(n^{\log_2 4}) = O(n^2)$

## Example 3

- $T(1) = 1$
- $T(n) = 2T(n/2) + n$

$a = 2; b = 2; a = b$

$T(n) = O(n \log n)$ If $a = b$

$T(n) = O(n \log n)$

## Example 2

$T(1) = 0$

$T(n) = T(n/2) + n$

$a = 1; b = 2; a < b$

$T(n) = O(n)$ if $a < b$

$T(n) = O(n)$

# Homework

استنتج العلاقات العودية لكل من الخوارزميات التالية وحللها بالطرق الثلاث -إن أمكن-ثم اعط تصورا عن تعقيدها:

الترتيب السريع والتريب بالدمج وابراج هانوي وفيبوناتشي والبحث الثنائي